

Experiment 4 | Lab Report

DHT11 Sensor Data Reading for Temperature and Humidity

Reading a digital temperature and humidity sensor over a single GPIO pin — printed on a Raspberry Pi terminal, and served by an ESP32 as a web page

Submitted by	Chavan Atharva Sunil
Roll Number	240003021
Date of Experiment	10 September 2026 (Raspberry Pi) and 3 September 2026 (ESP32)
Boards Used	Raspberry Pi 4 (GPIO2, physical pin 3) and ESP32 DevKit (GPIO4)

1 Aim

To measure **temperature** and **relative humidity** with a **DHT11** sensor connected to a GPIO pin of a **Raspberry Pi**. The sensor delivers both quantities as a digital 40-bit frame over a single data wire. In **Part A** a program on the Pi requests a reading every two seconds, decodes the frame, checks it, and prints the temperature in °C and the humidity in % to the terminal. In **Part B** the same sensor is read by an **ESP32**, which hosts its own Wi-Fi network and serves the two values as a web page to a phone.

Objectives

- To connect a DHT11 sensor module to the Raspberry Pi using only three wires — supply, ground and one data line on a GPIO pin.
- To understand the sensor's single-wire protocol: the start signal sent by the Pi, the sensor's response, and the 40-bit data frame with its checksum.
- To read the sensor from Python with the `adafruit_dht` library and print a continuous stream of temperature and humidity values.
- To observe how the readings behave over time, including the occasional failed read that the protocol's checksum detects.
- To read the same sensor from an ESP32 with the Arduino DHT library and publish the values over Wi-Fi as a web page, so that they can be viewed on a phone.

Brief Theory

The DHT11 is a low-cost digital sensor that combines a *capacitive humidity element*, an *NTC thermistor* and a small 8-bit microcontroller in one package. The microcontroller measures both elements, applies the factory calibration stored in its memory, and sends the result out as a digital word — so the host receives calibrated numbers rather than an analogue voltage that would need an ADC. The sensor covers 20–90 % relative humidity with an accuracy of $\pm 5\%$ and 0–50 °C with an accuracy of $\pm 2\text{ }^{\circ}\text{C}$, and it should be polled no faster than once per second (the library used here waits two seconds between reads).

Communication uses one bidirectional **single-wire** line, held HIGH by a pull-up resistor when idle. The host starts a reading by pulling the line LOW for at least 18 ms and then releasing it; the sensor answers with an 80 μs LOW followed by an 80 μs HIGH, and then sends **40 bits**. Every bit begins with a 50 μs LOW; the length of the HIGH pulse that follows encodes the value — about 26–28 μs for a 0 and 70 μs for a 1. The 40 bits are five bytes:

$$\underbrace{\text{RH}_{\text{int}}}_{\text{byte 1}} \underbrace{\text{RH}_{\text{dec}}}_{\text{byte 2}} \underbrace{\text{T}_{\text{int}}}_{\text{byte 3}} \underbrace{\text{T}_{\text{dec}}}_{\text{byte 4}} \underbrace{\text{checksum}}_{\text{byte 5}}, \quad \text{checksum} = (\text{byte}_1 + \text{byte}_2 + \text{byte}_3 + \text{byte}_4) \bmod 256.$$

The host measures the pulse widths, assembles the bytes and verifies the checksum; if the sum does not match, the frame is discarded and the read is repeated. Because the timing is in microseconds while the Pi runs a multitasking operating system, a fraction of reads are always lost this way — which is exactly what the program’s error messages report.

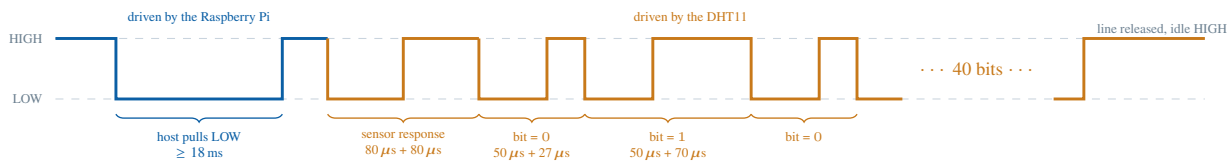


Figure 1: The DHT11 single-wire protocol (not to scale). The Pi sends the start signal; the sensor answers and then clocks out 40 bits, each distinguished by the width of its HIGH pulse. The last byte is a checksum of the other four.

Note on the module used. The sensor came as a three-pin **HW-481** breakout module, which carries the DHT11 together with its 10 kΩ pull-up resistor on the data line, so no external resistor was needed. The data pin was connected to **GPIO2** (physical pin 3). This pin doubles as the I²C SDA line, so the I²C interface must be disabled in raspi-config for the pin to be free as a GPIO; it also carries the Pi’s own fixed 1.8 kΩ pull-up to 3.3 V, which the DHT11 tolerates. On the ESP32 any free GPIO can be used; GPIO4 was used here.

2 Materials Required

#	Component	Qty.	Purpose in the setup
1	Raspberry Pi 4 (in case, with SD card, OS installed)	1	Part A host; drives the single-wire bus and prints to the terminal
2	ESP32 DevKit (30-pin)	1	Part B host; reads the sensor and serves the web page over Wi-Fi
3	DHT11 sensor module, HW-481 (3-pin, with pull-up)	1	Measures temperature and relative humidity; shared by both parts
4	Female–female jumper wires	3	Supply, data and ground between the header and the module
5	USB-C power supply, monitor, keyboard and mouse	1	Running and observing the Raspberry Pi
6	Micro-USB cable, laptop with Arduino IDE, and a phone with Wi-Fi	1	Programming the ESP32 and viewing its page
7	Software: adafruit-circuitpython-dht + libgpiod2 (Pi); DHT sensor library by Adafruit (Arduino)	—	Libraries that implement the DHT11 protocol

3 Methodology

No breadboard was needed: the module’s three pins were connected straight to the board’s header with three jumper wires — supply to 3.3 V, ground to GND, and the signal pin to one GPIO. **Part A** used the Raspberry Pi: the Adafruit DHT library was installed, and a short Python program creates a DHT11 object on board.D2, then loops forever reading temperature and humidity every two seconds and printing them. Failed reads — which the library reports as a RuntimeError when the checksum does not match or the frame is incomplete — are caught, printed, and simply retried on the next cycle. **Part B** moved the same module to an ESP32: an Arduino sketch reads it with the Adafruit DHT library, starts the ESP32 as a Wi-Fi access point, and answers web requests with a page containing the current values, which was opened on a phone.

3.1 Connections

Module pin	Raspberry Pi (Part A)	ESP32 (Part B)
+ (VCC)	Pin 1 — 3.3 V (red)	3V3
S (signal)	Pin 3 — GPIO2 (purple)	GPIO4
– (GND)	Pin 6 — GND (black)	GND

Numbering used here

The board module of CircuitPython names pins by their **BCM** GPIO number, so board.D2 is GPIO2, which sits at *physical* pin 3 of the 40-pin header — the second pin on the inner row, right next to the 3.3 V pin used for the supply. The three connections therefore all fall within the first three rows of the header.

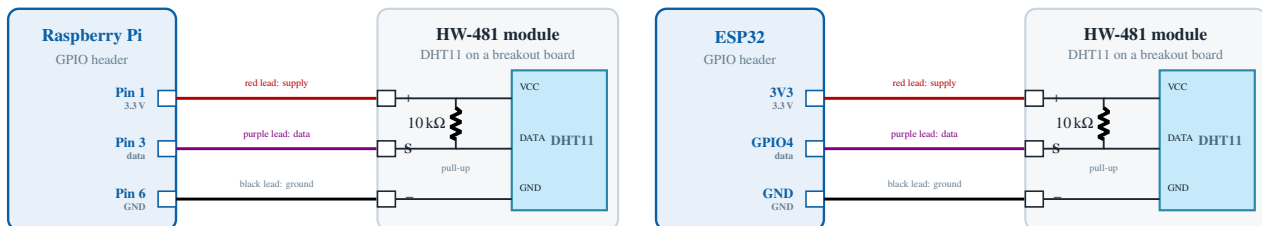


Figure 2: Circuit schematics for Part A (left) and Part B (right). The HW-481 module needs only supply, ground and one data line; its on-board 10 kΩ resistor holds the data line HIGH when neither side is driving it.

3.2 Part A — Raspberry Pi, readings on the terminal

The `adafruit_dht` library implements the microsecond-level protocol of Figure 1 in native code, so the Python program only has to ask for a value. It was installed on Raspberry Pi OS with:

Listing 1: Installing the DHT library on Raspberry Pi OS.

```
1 sudo apt install libgpiod2
2 pip3 install adafruit-circuitpython-dht
```

The library depends on Adafruit *Blinka*, which provides the board module that maps names such as D2 to the Pi's GPIO pins. The program itself is Listing 2.

Listing 2: Reading the DHT11 every two seconds and printing temperature and humidity.

```
1 import time
2 import board
3 import adafruit_dht
4
5 # HW-481 (DHT11) signal pin connected to GPIO2 = physical pin 3
6 dht = adafruit_dht.DHT11(board.D2)
7
8 try:
9     while True:
10         try:
11             temperature = dht.temperature # degrees Celsius
12             humidity = dht.humidity # percent relative humidity
13             print(f"Temperature: {temperature:.1f} °C")
14             print(f"Humidity: {humidity:.1f} %")
15             print("-" * 28)
16         except RuntimeError as error:
17             # A failed frame (bad checksum, incomplete data) is normal for
18             # a DHT sensor on a non-real-time OS: report it and try again.
19             print("DHT reading error:", error.args[0])
20             time.sleep(2.0) # DHT11 needs >= 1 s between reads
21
22 except KeyboardInterrupt:
23     print("\nExiting program")
24
25 finally:
26     dht.exit() # release the pin and the sensor
```

3.3 Part B — ESP32, readings on a web page

On the ESP32 the sensor is read with the Arduino *DHT sensor library* by Adafruit (installed from the IDE's Library Manager), whose `readTemperature()` and `readHumidity()` return NaN when a frame fails its checksum. Rather than

printing to the serial monitor, the sketch (Listing 3) turns the ESP32 into a Wi-Fi **access point** named ESP32_DHT11 and runs a web server on it. Every request for / triggers a fresh read of the sensor and returns a small HTML page with the two values — or “Sensor Error!” if the read failed — and the page asks the browser to reload itself every two seconds. A phone joined to the ESP32’s network and opened <http://192.168.4.1>, the access point’s address, which the sketch also prints on the Serial Monitor at start-up.

Listing 3: Part B — ESP32 sketch: the DHT11 readings served as a web page from the board’s own access point.

```

1  #include <WiFi.h>
2  #include <WebServer.h>
3  #include <DHT.h>
4
5  #define DHTPIN 4           // DHT11 signal pin
6  #define DHTTYPE DHT11
7
8  const char *AP_SSID = "ESP32_DHT11"; // the ESP32 creates this Wi-Fi network
9  const char *AP_PASS = "12345678";
10
11 DHT dht(DHTPIN, DHTTYPE);
12 WebServer server(80);
13
14 void handleRoot() {          // called for every request of "/"
15     float temperature = dht.readTemperature(); // degrees Celsius, NaN on a bad frame
16     float humidity = dht.readHumidity();        // percent
17
18     String html = "<!DOCTYPE html><html><head><meta charset='utf-8'>";
19     html += "<meta http-equiv='refresh' content='2'>"; // reload every 2 s
20     html += "<title>ESP32 DHT11</title></head><body>";
21     html += "<h1>ESP32 DHT11 Sensor</h1>";
22
23     if (isnan(temperature) || isnan(humidity)) {
24         html += "<h2>Sensor Error!</h2>";
25     }
26     else {
27         html += "<h2>Temperature: ";
28         html += String(temperature);
29         html += " &deg;C</h2>";
30
31         html += "<h2>Humidity: ";
32         html += String(humidity);
33         html += " %</h2>";
34     }
35
36     html += "</body>";
37     html += "</html>";
38
39     server.send(200, "text/html", html);
40 }
41
42 void setup() {
43     Serial.begin(115200);
44     dht.begin();
45
46     WiFi.softAP(AP_SSID, AP_PASS); // start the access point
47     Serial.println("Wi-Fi started!");
48     Serial.print("Network: ");      Serial.println(AP_SSID);
49     Serial.print("ESP32 IP address: "); Serial.println(WiFi.softAPIP()); // 192.168.4.1
50
51     server.on("/", handleRoot);
52     server.begin();
53     Serial.println("Web server started!");
54 }
55
56 void loop() {
57     server.handleClient();
58 }

```

3.4 Procedure

1. **Part A.** With the Raspberry Pi powered off, the module’s +, S and – pins were connected with three jumper wires to pin 1 (3.3 V), pin 3 (GPIO2) and pin 6 (GND) of the header, and the connections were checked against the pinout.
2. The Pi was booted, the I²C interface was confirmed to be disabled, and the library of Listing 1 was installed.
3. The program of Listing 2 was saved as dht.py and started from the terminal with `python3 dht.py`.
4. The terminal was observed while the program printed a new temperature and humidity pair every two seconds; the readings were recorded on video, together with the error lines produced whenever a frame failed its checksum.
5. The program was stopped with Ctrl+C, which releases the sensor and the GPIO pin.
6. **Part B.** The module was moved to the ESP32 (+ to 3V3, – to GND, S to GPIO4). In the Arduino IDE the ESP32

board package and the Adafruit DHT library were installed, Listing 3 was uploaded to the board on its COM port, and the Serial Monitor showed the network name and the address 192.168.4.1.

- A phone was connected to the ESP32_DHT11 network and the address opened in its browser; the page showed the temperature and humidity and refreshed itself every two seconds. This was recorded on video.

4 Result and Output

The sensor was read successfully over the single data wire on both boards. In **Part A** the Raspberry Pi printed a temperature and humidity pair every two seconds; over the recorded run of **18 readings** the temperature fell steadily from 30.9 to 28.7 °C while the relative humidity rose from 46 to 53 %. Four attempts in the same run failed the checksum or returned an incomplete frame; each was reported by the program and the next read succeeded. In **Part B** the ESP32 served its page at 192.168.4.1, and the phone showed **26.10 °C and 55.80 %** in the lab.

Observed outputs — Part A, Raspberry Pi terminal

#	Temp. (°C)	RH (%)	Remark	#	Temp. (°C)	RH (%)	Remark
1	30.9	46		10	29.6	50	
2	30.9	46		11	29.4	50	then 1 checksum error
3	30.8	47		12	29.3	51	
4	30.6	47		13	29.1	51	
5	30.5	47		14	29.1	51	
6	30.3	48		15	29.0	52	
7	30.2	48		16	28.8	52	
8	30.1	48	then 2 checksum errors	17	28.8	52	
9	29.8	49	then 1 incomplete frame	18	28.7	53	

The 18 values were transcribed from the terminal as recorded in the two video clips, which are consecutive (the last reading visible in the first clip is the first in the second). With the program's two-second interval the run spans roughly 45 s including the four failed attempts.

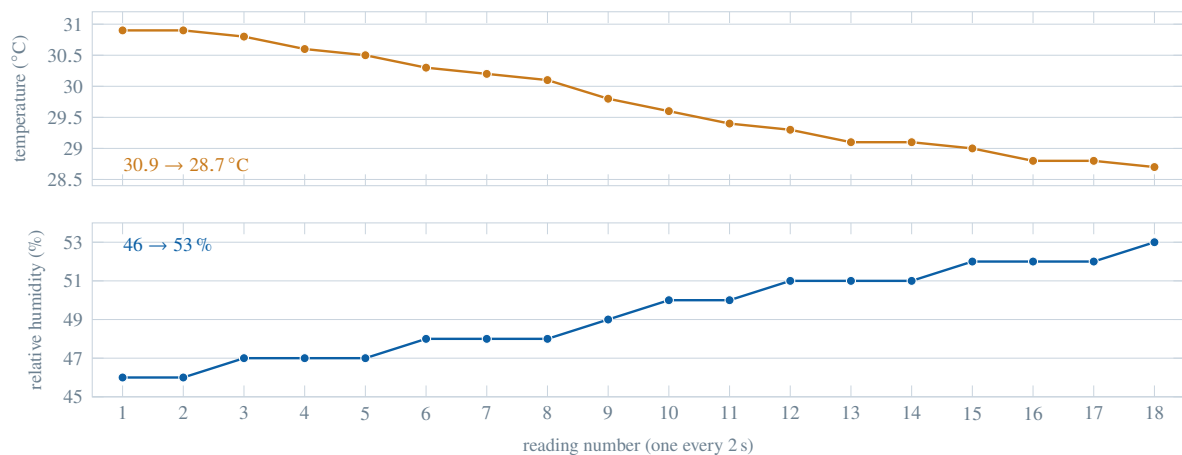
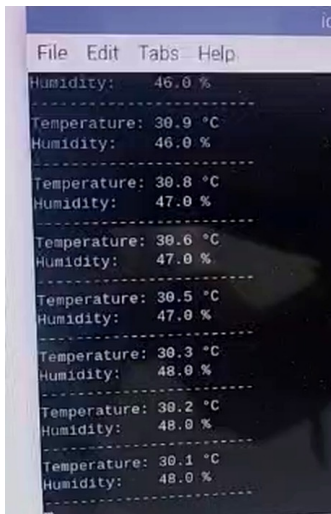
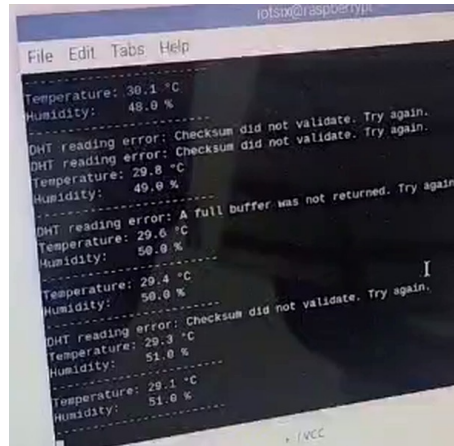


Figure 3: The 18 recorded readings. Temperature (top) drifts down by 2.2 °C over the run while relative humidity (bottom) climbs by 7 % — the two move in opposite directions, as expected for air whose moisture content is not changing (see Discussion). The four failed attempts fell between readings 8–9, 9–10 and 11–12 and do not appear as points.

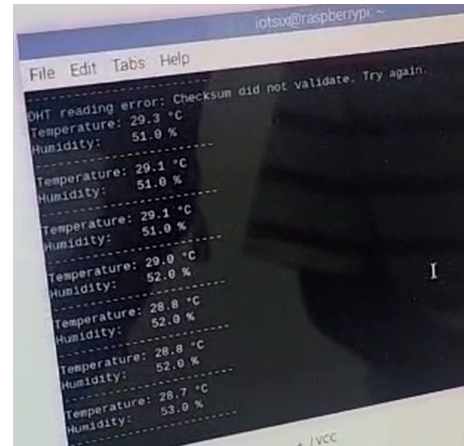
Recorded outputs — Raspberry Pi terminal



readings 1–8



readings 8–13, with the four failed reads



readings 12–18

Figure 4: Three frames from the recording of the terminal on the Raspberry Pi while `dht.py` was running. Each successful read prints a temperature and a humidity line followed by a rule; a failed read prints DHT reading error: with the library’s message — *Checksum did not validate* when the fifth byte does not match the other four, and *A full buffer was not returned* when fewer than 40 bits were captured.

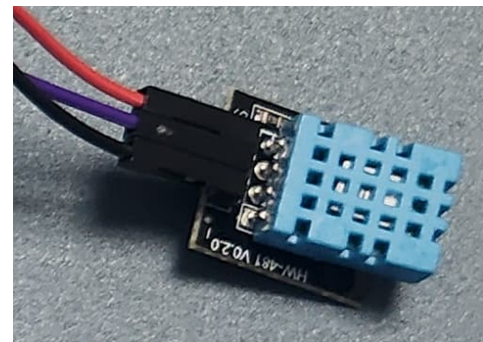
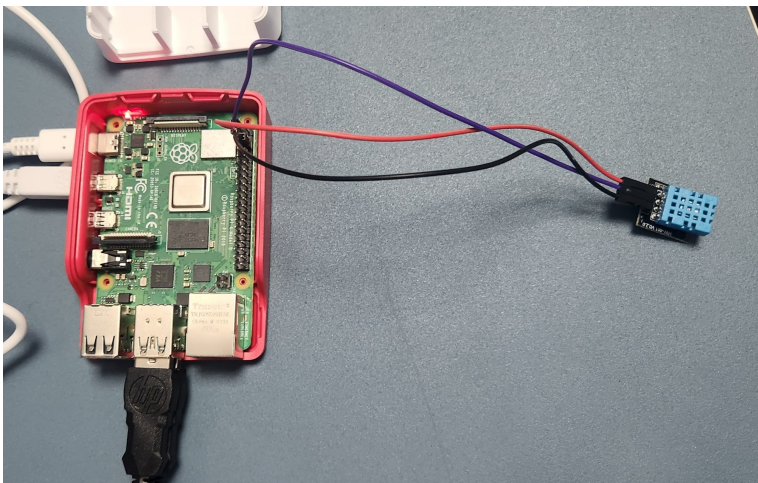


Figure 5: *Left:* the Raspberry Pi 4 with the DHT11 module hanging off three jumper wires — red to 3.3 V, purple to GPIO2, black to GND — entering the first rows of the GPIO header. *Right:* the HW-481 module (marked “HW-481 V0.2.0”) carrying the blue DHT11 and its pull-up resistor.

Observed output — Part B, ESP32 web page

Output	Where	Value shown	Remark
4	Phone browser at http://192.168.4.1	26.10 °C, 55.80 %	page titled “ESP32 DHT11 Sensor”, served by the ESP32 access point ESP32_DHT11; refreshes every 2 s
5	Arduino Serial Monitor	ESP32 IP address: 192.168.4.1	printed by the sketch after Wi-Fi started! and before Web server started!

The ESP32 run took place in the air-conditioned lab on a different day from Part A, which is why its reading (26.1 °C, 55.8 %) does not match the Pi’s; the DHT11’s specified accuracy is $\pm 2^\circ\text{C}$ and $\pm 5\%$ RH in any case.

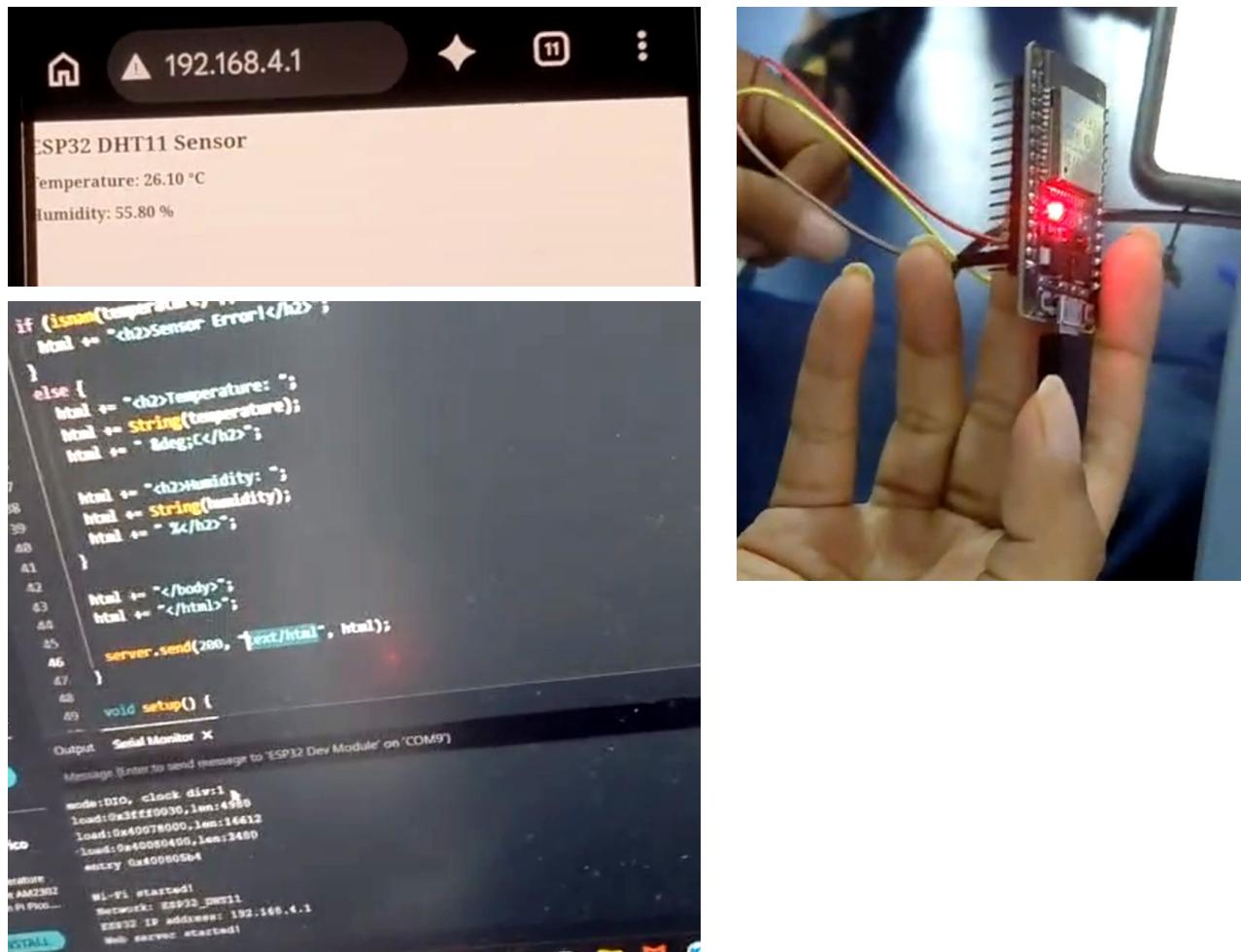


Figure 6: Part B. *Top left:* the page served by the ESP32, opened on a phone at 192.168.4.1 — “Temperature: 26.10 °C, Humidity: 55.80 %”. *Bottom left:* the Arduino IDE with the sketch’s page-building code and, in the Serial Monitor, the start-up messages Wi-Fi started!, Network: ESP32_DHT11, ESP32 IP address: 192.168.4.1, Web server started!. *Right:* the ESP32 DevKit with the DHT11 module on three jumper wires.

Video demonstration and code

Google Drive link

The demonstration videos, this report and the complete source code are available in the following shared folder:

https://drive.google.com/drive/folders/11TP-NX_EF_IpxHIAUYmG8YWPQfst2ae2?usp=sharing

Access has been set to *anyone with the link can view*.

Discussion

- **Why the readings drift.** The temperature fell by 2.2 °C and the humidity rose by 7 % over less than a minute. The sensor had just been handled while wiring it, so it started warmer than the room air and cooled towards it. Relative humidity is the ratio of the water vapour present to the maximum the air can hold at that temperature, and the maximum falls as the air cools; with the same moisture content, RH therefore rises as the temperature drops — exactly the opposite-going curves of Figure 3.
- **Failed reads are expected.** Four of 22 attempts failed. The bit timing of the protocol is measured in tens of microseconds, but Raspberry Pi OS is not a real-time system and can pre-empt the reading routine at any moment, stretching a pulse or losing the end of the frame. The checksum catches the corrupted frames, and the correct response is simply to discard them and read again — which the program does by catching the `RuntimeError`.
- **Resolution and accuracy.** The values are printed to one decimal place but the DHT11 is only specified to $\pm 2^\circ\text{C}$ and $\pm 5\%$ RH, and the humidity byte here changes only in whole percent. The decimal place is useful for seeing the *trend*, not as an absolute accuracy. For better figures the pin-compatible DHT22 ($\pm 0.5^\circ\text{C}$, $\pm 2\%$) can be swapped in by changing one line of the program.

- **The single-wire interface.** Unlike the earlier experiments, in which the GPIO pin was only ever an output, here the same pin is driven by the Pi for the start signal and then switched to an input to listen to the sensor. Two pull-ups end up in parallel on the line — the module's 10 k Ω and the Pi's 1.8 k Ω on GPIO2 — giving about 1.5 k Ω , which the DHT11 can still pull LOW. On a pin without a board pull-up (say GPIO4) only the module's resistor would be present.
- **Sampling rate.** The two-second delay is not just politeness: the DHT11 needs at least a second between conversions, and reading it faster returns stale or invalid data. On the ESP32 the page's two-second refresh serves the same purpose, since each page load triggers one read.
- **Serving the data instead of printing it.** The ESP32 has no screen, but it has Wi-Fi. Running it as an access point means the phone talks to it directly, without any router, at the fixed address 192.168.4.1; the web page is built as a string on every request, so it is always current. The Arduino library signals a failed frame by returning NaN rather than raising an exception, which the sketch turns into the "Sensor Error!" line instead of printing nonsense values.

5 Conclusion

The experiment moved from driving outputs to **acquiring data**: a DHT11 sensor was read from a single GPIO pin — of a Raspberry Pi, which printed a fresh pair of calibrated temperature and humidity values every two seconds, and of an ESP32, which served the same two values as a web page over its own Wi-Fi network to a phone. Eighteen consecutive readings showed the sensor settling from 30.9 to 28.7 °C with humidity rising from 46 to 53 %, the two quantities moving in opposite directions as the physics of relative humidity predicts, and the four frames that failed the checksum were detected and discarded rather than reported as false values.

The exercise showed the practical side of a digital sensor interface — installing a driver library, the meaning of its error messages, and the timing limitations of a general-purpose operating system — and, with the ESP32, took the first step from reading a sensor to *publishing* its data over a network, which is what an IoT node does.